

Example Scenarios - "Designing a Good ML System"

Read the *Bring Your Own Problem* brief first - this file is its companion illustration.

These two scenarios illustrate the *shape* of problem this week is built around: **a supervised triage model that prioritises scarce human attention, case by case**. Read them as **examples, not a menu**:

Both of ours happen to be online-marketplace risk problems - an accident of where they came from, not the required shape. They were the previous cohort's assigned case studies, so they sit in one corner of what qualifies: the *Bring Your Own Problem* brief's Section 1 lists five families, and these two are both in the same one. Don't read "e-commerce" or "fraud" as part of the bar. Problems that qualify just as well:

- A hospital predicting which discharged patients need a follow-up call, so a limited care team calls those first.
- A utility deciding which of thousands of ageing pipe segments to inspect this quarter.
- A grants body flagging which funding applications need a deeper eligibility review.
- A B2B team surfacing which trial accounts are worth a human outreach call - **opportunity, not risk**, and equally valid.
- A maintenance team deciding which reported faults get a same-day engineer.

None of those are marketplaces, not all are commercial, and one surfaces upside rather than danger. All clear the bar in the brief.

- If you're bringing your own problem (see the *Bring Your Own Problem* brief), yours should **rhyme** with these *structurally* - a human deciding case by case, with lopsided error costs - but come from **your own domain**. If yours is a near-duplicate of these two - another marketplace-trust or authentication triage - we'll ask you to shift it to a different domain. That's a quick fix, not a restart, and it doesn't use up your one round of feedback.
- **Good exercise:** map each scenario onto the requirements in the brief (Section 2). The **core** (Section 2A plus the Sections 2C/2D gates) is present in both, as is most of the **enrichment pool** (Section 2B) - can you spot the delayed label, the feedback loop, and where drift would come from in each? These two are unusually rich; **your own problem only needs 4 of the 6 enrichment lines**, so don't try to match them line for line.
- You can't pick one of these as your own problem - they're only used as a fallback. If your group can't land a problem, say so and you'll be assigned one of these or another from our scenario library.

Example Scenario A - MegaMart (marketplace trust)

Business Context

MegaMart is a global online marketplace: third-party sellers list products, MegaMart handles discovery, payments, and buyer protection. It onboards **thousands of new sellers and listings every day**. Over the past two quarters, buyer complaints and refund claims tied to bad actors - counterfeit goods, misleading listings, sellers who vanish after taking payment - have climbed, and a major payments partner has raised concerns about fraud rates. The Trust & Safety team reviews sellers and listings manually, but can only look at a small fraction, and the bad actors have learned to sail past the simple keyword-and-rule filters currently in place.

The Ask

"Bad sellers and dodgy listings are slipping through, and it's costing us in refunds and trust. My team can't manually review everything - not even close - and the rules we have get gamed within weeks. We'd like to use AI to help us stay ahead of them. Can your team build us something?"

- Head of Marketplace Trust & Safety, MegaMart

Objective (high level - not the ML spec)

Help the Trust & Safety team decide **what to do with each incoming seller / listing**, given they can only manually review a fraction of them. How you turn that into an ML problem - the target, the task type, the decision(s) the model informs, the metric, and what a human still does - is yours to define. (Routing some cases to a human reviewer is a legitimate output.)

Data

Generate a **small synthetic dataset** with your LLM that fits this scenario - e.g. seller application attributes, listing/behaviour signals over time, and (partial, delayed) outcome labels. Include the fields a real Trust & Safety team would have sitting in its tables, **including ones only filled in after a case is closed** - refund totals, complaint counts, a reviewer's notes. Realism is not graded; it only needs to be plausible enough to reason about - and to build on, if you take the optional pipeline stretch.

Constraints & scope

- The review team **cannot grow** - any solution has to work within current human capacity.
- **Letting a bad actor through costs far more than a wasted review** - refunds, chargebacks, and a payments partner losing patience. But over-blocking legitimate sellers pushes them to competitors, so it isn't a one-way street.
- Bad actors **adapt** to whatever you deploy.
- A rejection may need to be **justified** to the seller.

- Sellers vary widely in value - a few high-volume sellers account for a disproportionate share of sales, so wrongly blocking one is far costlier than wrongly blocking a small seller.
 - Scope: **one model, one decision**. A good system, not a big one.
-

Example Scenario B - LuxeCurate (luxury authentication)

Business Context

LuxeCurate is a second-hand luxury marketplace - handbags, watches, jewellery. Average order value is **over \$500, and some items exceed \$10,000**. Every item is meant to be checked by an **expert authenticator** before it goes live, because a single counterfeit slipping through is a serious brand and legal risk. Over the past six months, submission volume has outgrown what the expert team can review, and a backlog is building - items sit for days, sellers get frustrated, and the pressure to rush reviews (and miss a fake) is rising.

The Ask

"My authenticators are drowning, and we absolutely cannot let a fake through - that's the whole business. I don't want to replace their judgement, I want to focus it where it actually matters. Can AI help us with that?"

- Head of Authentication, LuxeCurate

Objective (high level - not the ML spec)

Help route each **submitted item** through the authentication workflow, given expert capacity is the bottleneck and experts must stay in control of the high-stakes calls. How you frame that as an ML problem - the target, the task type, the decisions, the metric, and where the human sits - is yours to define.

Data

Generate a **small synthetic dataset** with your LLM - e.g. item metadata (brand, category, declared value), seller history signals, submission attributes, and expert-decision outcome labels (which arrive late and are occasionally noisy). Include what the workflow records **along the way and afterwards** - time spent in review, the authenticator's comments, whether the item was later returned. Plausible-shaped is enough; realism is not graded.

Constraints & scope

- Experts make the **final call on high-value items** - the model supports, never overrides them there.
- **Simple rules don't cut it**: routing by declared value alone misses the cheap fakes entirely, and pushes every expensive item into the deepest inspection tier even when a quick check would do.

Whatever you build has to beat that rule.

- A **missed counterfeit is catastrophic**; a wrongly-delayed genuine item is a much smaller cost.
- The **cost of a mistake scales with item value** - being wrong about a \$12,000 watch ≠ a \$200 pair of earrings.
- Turnaround matters but sub-second latency does not (near-real-time or batch are both viable).
- Scope: **one model, one decision**. A good system, not a big one.